

أهلاً بك يا بني، وزميلي المهندس المبدع. يسعدني جداً أن أرى ثمار تدريبنا الهندسي العميق (في مشاريع Upwork السابقة) تتجلى الآن في قدرتك على اصطيد فرص حقيقية وعالية القيمة في سوق العمل الحر. هذا العرض (Job Post) يبحث عن "مهندس برمجيات حقيقي" وليس مجرد "هاوي سكرابينغ". العميل يدرك تماماً تعقيد أمازون، ويدرك أن مكنتبات مثل Requests العادية ستفشل فوراً، وأن Selenium سيكون ثقيلاً جداً. بناءً على طلبك، سأقوم بتقسيم إجابتي إلى ثلاثة أقسام رئيسية:

1. صياغة العرض الاحترافي (Proposal) لتقديمه للعميل فوراً.
2. التحليل المعماري للمشروع وكيف سنطبق ما تعلمناه.
3. الكود البرمجي الهندسي (المنظومة) كملف بايثون جاهز.

أولاً: العرض الاحترافي (Proposal) للعميل

انسخ هذا النص وقم بتعديله قليلاً ليناسب أسلوبك، ثم أرسله للعميل: "فهمت طبيعة عروض أمازون"

مرحباً،

بصفتي مهندس برمجيات وأتمتة متخصص في هندسة البيانات (Data Engineering) وتجاوز أنظمة الحماية المعقدة (Anti-Bot Systems)، اطلعت على متطلبات مشروعك، وهي تتطابق تماماً مع هيكليات الأنظمة التي أقوم ببنائها. لمنصة مثل Amazon Egypt، استخدام أدوات مثل Selenium أو Playwright لعمليات الـ Monitor على مدار 24/7 هو خطأ استراتيجي يستنزف موارد الخادم (RAM/CPU) ويبطئ سرعة الاكتشاف. استراتيجيتي التقنية لتنفيذ هذه المنظومة:

1. المحرك الشبكي (Network Engine): سأعتمد على مكتبة curl_cffi (أو tls_client) لبناء طلبات HTTP متزامنة وغير متزامنة (Async) خفيفة جداً، مع عمل محاكاة كاملة لبصمة المتصفح (TLS/JA3 Fingerprint Spoofing) لتبدو الطلبات وكأنها من متصفح Chrome حديث، مما يمنع أمازون من حظر الطلب.
 2. الاستهداف الذكي (Smart Interception): لن نقوم بتحميل الـ DOM الثقيل، بل سنرصد نقاط النهاية (API Endpoints) وملفات الـ JSON المخفية في الواجهة الأمامية التي تغذي قسم العروض (Deals & Promotions) لاصطياد الـ Promotion ID في أجزاء من الثانية.
 3. هندسة الصمود (Resilience & Proxy Management): سأقوم بتضمين خوارزمية "قاطع الدائرة" (Circuit Breaker) ومقيد الطلبات التكيفي (Adaptive Rate Limiter) للتعامل بكفاءة مع الوكلاء (Proxies)، بحيث إذا تم حظر IP، يقوم النظام بالتبديل فوراً وتعديل زمن الطلبات دون أن ينهار السكربت.
 4. التنبيه اللحظي (Instant Alerting): دمج النظام مع Telegram Webhook/API لإرسال رسالة فورية (URL + اسم العرض) بمجرد تقاطع البيانات مع قاعدة البيانات المؤقتة (In-Memory Cache) التي تمنع تكرار الإشعارات.
- الأداة ستكون مكتوبة بـ Python، خفيفة جداً، جاهزة للعمل على خادم (VPS) صغير، ومستقرة تماماً. يسعدني مناقشة التفاصيل الفنية والبدء فوراً.

تحياتي،

ثانياً: التحليل المعماري والدراسة الفنية للمنظومة

- يا بني، هدفنا هنا هو "السرعة والخفة". أمازون لا تضع العروض الترويجية في الـ HTML الثابت، بل تأتي عبر طلبات AJAX. لماذا curl_cffi وليس Playwright؟ العميل يريد أداة تعمل 24/7 بسرعة فائقة. Playwright يستهلك حوالي 200-500 ميجابايت من الـ رام لكل متصفح، بينما curl_cffi يستهلك أقل من 30 ميجابايت لأنه يرسل طلبات HTTP خفيفة ولكنه يزور "بصمة التشفير" (TLS) ليبدو كمتصفح حقيقي.
1. إدارة الذاكرة (O(1\$): بما أن السكربت سيعمل لأيام وشهور، إذا قمنا بتخزين الروابط المكتشفة في List، سننفجر الذاكرة. سنستخدم Set (مجموعة) بحجم أقصى (مثلاً 1000 رابط) ونقوم بتفريغها برمجياً لضمان استقرار الـ رام.
 2. التعافي الذكي: سنطبق ما تعلمناه في مشروع Upwork؛ إذا ردت أمازون بخطأ 503 أو طلب CAPTCHA، سيقوم النظام

تلقائياً بتغيير البروكسي وتأخير الطلب القادم بضع ثوانٍ.

ثالثاً: الكود البرمجي للمنظومة (Amazon Promo Monitor)

لقد صممت لك هذه المنظومة في ملف واحد قوي ومستقل لتتمكن من تشغيله فوراً وتقديمه كنموذج عملي.

Amazon Promo Monitor

14 يونيو، 11:27 م

أريد إعادة المحاولة بدون استخدام "Canvas"

خطواتك القادمة كمهندس محترف:

1. تثبيت المكتبة الخارقة: الأداة تعتمد على `curl_cffi` وهي مكتبة تبدو للأنظمة وكأنها لغة C تتعامل مع المتصفح، مما يكسر حماية بصمة الـ TLS. قم بتثبيتها عبر الأمر:
`pip install curl_cffi`
2. ضبط التعبيرات النمطية (Regex): في دالة `extract_promotions`، قمت بوضع تعبير نمطي يتصيد الروابط التي تحتوي على `/promotion/psp/`. قد تحتاج لفحص واجهة أمازون مصر الحالية وتعديل الـ Regex قليلاً لالتقاط أنماط روابط إضافية.
3. التشغيل: السكربت جاهز. فقط ضع الـ Token الخاص بتليجرام وسيبدأ بالعمل كحارس لا ينام. أنا فخور برويتك تنتقل من مرحلة التعلم إلى مرحلة استهداف مشاريع برمجية حقيقية وحساسة. توكل على الله وقدم العرض، وأنا هنا بجانبك لضبط أي تفاصيل برمجية عند استلامك للمشروع!